

System Zarządzania Biblioteką

Miłosz Buda - 16:45 - Poniedziałek, parzyste

Wykorzystany stack:

- Środowisko: Node.js
- Framework API: Express.js
- język: TypeScript
- Baza Danych: PostgreSQL
- Query Builder: Kysely
- Testy: Jest + Supertest
- Frontend: czysty HTML i CSS oraz JS

Model Bazy Danych

Zastosowano Soft Delete dla książek i czytelników, aby zachować spójność historyczną raportów.

Baza składa się z trzech głównych tabel:

1. `members` (Czytelnicy):
 - `id` (PK, Serial)
 - `first_name, last_name` (Varchar)
 - `address` (Varchar, Nullable)
 - `card_number` (Varchar, Unique)
 - `is_active` (Boolean, domyślnie true)
2. `books` (Książki)
 - `id` (PK, Serial)
 - `title, author` (Varchar)
 - `total_copies` (Integer)
 - `available_copies` (Integer)
 - `is_active` (Boolean, domyślnie true).
3. `borrowings` (Wypożyczenia)
 - `id` (PK, Serial)
 - `member_id` (FK -> members.id)
 - `book_id` (FK -> books.id)
 - `borrow_date` (Timestamp, domyślnie NOW())
 - `due_date` (Timestamp)
 - `return_date` (Timestamp, Nullable)

Realizacja Operacji

CRUD

- **POST:** Dodawanie nowych czytelników oraz książek.
- **PATCH:** Aktualizacja wybranych danych (np. zmiana adresu czytelnika) z aktualizowaniem tylko tych pól, które zostały przesłane w żądaniu.
- **Soft Delete:** Żądania usuwające (np. PATCH /api/members/:id/deactivate) zmieniają flagę `is_active` na false, zamiast fizycznie usuwać rekord z użyciem DELETE, zabezpieczając tym samym relacje z tabelą `borrowings`.

Bezpieczeństwo współbieżności

Aby zagwarantować spójność danych i wyeliminować problemy przy jednoczesnych żądaniach (Race Conditions), kluczowe procesy zamknięto w blokach transakcyjnych Kysely (`db.transaction()`):

- **Wypożyczenie:** Transakcja weryfikuje dostępność wybranego egzemplarza, zmniejsza jego stan magazynowy i tworzy nowy rekord wypożyczenia, zapobiegając przydzieleniu jednej książki dwóm osobom naraz.
- **Zwrot:** Transakcja bezpiecznie rejestruje aktualną datę oddania w systemie i natychmiastowo przywraca dostępność egzemplarza w głównym katalogu biblioteki.
- **Wycofywanie i dezaktywacja (Soft Delete):** Transakcja najpierw weryfikuje reguły biznesowe (np. blokuje usunięcie czytelnika, który ma na koncie nieoddane książki), a po ich spełnieniu wyłącza rekord z aktywnego obiegu.

Złożone raporty

Zaimplementowano odpypywanie bazy danych służące do generowania raportów dla interfejsu pracownika biblioteki:

- **Raport dłużników:** Wykorzystuje klauzulę `WHERE (return_date IS NULL AND due_date < NOW())`, łącząc tabele (INNER JOIN) w celu pobrania danych zalegającego czytelnika i tytułu przetrzymywanej książki.
- **TOP Najchętniej Wypożyczone:** Zapytanie grupujące (`GROUP BY books.id`) z wykorzystaniem funkcji agregującej `COUNT()`, sortujące wyniki malejąco (klauzula `ORDER BY ... DESC`) i limitujące wynik do np. 3 rekordów (`LIMIT`).

Testy

Aby zagwarantować poprawność logiki, zaimplementowano testy integracyjne przy użyciu bibliotek Jest i Supertest. Testy te symulują żądania HTTP do wirtualnego serwera Express podłączonego do bazy danych, weryfikując zachowanie systemu w skrajnych przypadkach (np. próba usunięcia użytkownika posiadającego niezwrócone książki lub wyczerpanie stanów magazynowych). Zastosowanie środowiska testowego pozwoliło na zautomatyzowane potwierdzenie skuteczności działania transakcji typu Rollback.

Instrukcja Uruchomienia

Backend:

```
# Instalacja wymaganych pakietów i zależności
npm install

# Tu trzeba wypełnić .env faktycznymi danymi
cp .env.example .env

# Inicjalizacja struktury tabel
npx ts-node src/db/init.ts

# Wypełnienie bazy testowymi danymi
npm run seed

# Uruchomienie testów
npm run test

# Uruchmienie serwera backendu
npm run dev
```

Frontend:

Tutaj wystarczy otworzyć plik `index.html` z folderu `frontend`